Penetration Testing with Large Language Models

Student introduction and lab guide

Your task

Use an authorized local Docker lab to learn how a tester observes a web application, forms a hypothesis, checks it safely, and records one final flag. ChatGPT, DeepSeek, or another chatbot may help you reason through the evidence.

What penetration testing means

Penetration testing is an authorized security assessment in which a tester behaves like an attacker to find practical ways around an application's protections. The goal is to discover weaknesses before a real attacker does, explain their impact, and support a fix. It is more focused than randomly trying attacks: every action should stay within an agreed scope and produce evidence that another person can review.

A penetration test examines how several parts of a system work together. A tester may review visible pages, input fields, URLs, login behavior, access-control decisions, error messages, and network requests. Finding unusual behavior is only the start: the tester must check whether it is repeatable, decide what information or capability is exposed, and describe how the weakness could be corrected.

What a basic test includes

Stage	Purpose
Scope and rules	Define the target, allowed actions, time, and evidence to collect.
Discovery	Map pages, inputs, identities, and requests before choosing a test.
Controlled testing	Change one condition at a time and compare the observed result.
Reporting	Explain the weakness, evidence, impact, and a possible mitigation.

Why we do it

- Find weaknesses that ordinary functional testing may miss.
- Show whether a weakness can cause real unauthorized access or data exposure.
- Give developers clear evidence so they can reproduce and fix the problem.
- Practice security thinking in a controlled environment.

A simple example

Imagine that a student portal displays your profile using /profile?id=101. You change only the number to 102. If the server returns another student's profile without checking ownership, you have found a broken access-control issue. A useful report records the original request, the changed value, the response, and why the result matters. It does not continue collecting unrelated data.

Define scope	Observe	Form a hypothesis	Test safely	Record evidence

Authorization comes first

For this session, test only the lab running on your own computer at 127.0.0.1. Do not apply these techniques to university systems, public websites, or any system for which you do not have explicit permission.

Using large language models during penetration testing

Traditionally, a human tester maps the target, inspects requests and responses, recalls possible vulnerability classes, chooses each next test, and keeps notes. Those skills still matter. A large language model can help organize evidence and explain unfamiliar ideas, while the student decides what is safe and whether the result is correct.

Manual workflow	LLM assisted workflow
Map pages and requests by hand	Ask the model to organize observations
Recall possible vulnerability classes	Ask for hypotheses linked to evidence
Interpret responses and errors	Ask for a plain-language explanation
Choose and document the next test	Request a short list of safe next checks

What an LLM can help with

- Explain an HTTP request, response, error message, cookie, or URL parameter.
- Suggest a few hypotheses that match the evidence already collected.
- Turn a hypothesis into a small, safe test inside the local lab.
- Summarize findings and help write clear reproduction steps.

How LLM assisted testing works

An LLM-assisted workflow keeps a clear goal, a history of evidence, and a short plan. The model proposes one next task, a person or authorized tool performs that task, and the result returns to the plan. Research systems add components such as separate planning agents, retrieved security knowledge, and summaries of earlier steps. In this lab, your browser is the testing tool, the chatbot helps with reasoning, and you validate every result.

Set the goal	Plan one step	Run a safe check	Interpret evidence	Human validates

Suggested reading on automated testing with LLMs

- **PentestGPT** — organizes reasoning and evidence across testing steps. Read the paper
- **PentestAgent** — uses multiple agents and retrieved security knowledge. Read the paper
- **HackSynth** — pairs a planner and summarizer for CTF tasks. Read the paper

A useful prompt pattern

```
I am working only on a local CTF at http://127.0.0.1:8080.
I observed: [paste the URL, request, response, or error].
Explain what this may mean and suggest up to three safe next checks.
For each check, tell me what result would support the hypothesis.
```

Treat the chatbot as a reasoning partner

LLMs can invent routes, commands, or explanations. Compare each suggestion with what the lab shows. Share only material from this fictional local lab, never personal passwords, tokens, or unrelated private data.

Complete the Campus Help Desk CTF

You need Docker Desktop or Docker Engine with Docker Compose. Keep README.md and compose.yaml in the same folder. The Compose file downloads the lab image automatically.

1. **Start the lab.** Open a terminal in that folder and run:

```
docker compose up -d
```

2. **Open the website.** Visit http://127.0.0.1:8080 and select Start lab.
3. **Explore before guessing.** Read the pages, inspect links and URLs, and notice how the browser communicates with the server. Browser developer tools are optional.
4. **Ask a chatbot for help.** Provide your current observation and ask for an explanation or a few safe next tests. You may paste text or upload a screenshot from the local lab.
5. **Test one idea at a time.** Change a single input, observe the response, and keep short notes. Use the optional hints page if you become stuck.
6. **Recover the final flag.** Follow the connected story until the recovery document displays one value in this form:

```
CTF{final_<32 lowercase hex ID>_<32 lowercase hex tag>}
```

7. **Submit and stop.** Copy the complete flag exactly. Then stop the lab with:

```
docker compose down
```

Useful commands

Situation	Command or action
Port 8080 is busy	CTF_PORT=8081 docker compose up -d, then open 127.0.0.1:8081
Restart without changing your flag	docker compose down, then docker compose up -d
Reset and generate a new flag	docker compose down -v, then docker compose up -d

Before you submit

- Confirm that the flag begins with CTF{final_ and includes the closing brace.
- If process evidence is requested, keep a few screenshots and brief notes showing your reasoning.
- Be ready to explain one observation, one hypothesis, and how the result supported it.

References

1. NIST SP 800-115, Technical Guide to Information Security Testing and Assessment. doi.org/10.6028/NIST.SP.800-115
2. Deng et al., PentestGPT: Evaluating and Harnessing Large Language Models for Automated Penetration Testing. usenix.org/conference/usenixsecurity24/presentation/deng
3. Shen et al., PentestAgent: Incorporating LLM Agents to Automated Penetration Testing. arxiv.org/abs/2411.05185
4. Muzsai et al., HackSynth: LLM Agent and Evaluation Framework for Autonomous Penetration Testing. arxiv.org/abs/2412.01778